

THE MATCOM MINIX INFORME TÉCNICO

Asignatura Sistemas Operativos
Facultad de Matemática y Computación
Universidad de La Habana

Instrucciones generales

- Lea toda la guía antes de comenzar a escribir.
- Rellene cada sección con el contenido solicitado.
- Utilice párrafos para explicar el qué y el porqué de cada decisión.
- Utilice viñetas solo para listar pasos, opciones o elementos concretos.
- Sea concreto, preciso y técnicamente riguroso.
- Incluya código, capturas o tablas cuando sea necesario para evidenciar su trabajo.
- Justifique siempre por qué eligió una solución frente a otras posibles.
- Extensión recomendada: entre 8 y 12 páginas (sin incluir anexos).

Integrantes

- Nombre y apellidos estudiante 1. Grupo
- Nombre y apellidos estudiante 2. Grupo

1. Introducción

La introducción debe contextualizar el proyecto dentro de la asignatura Sistemas Operativos, explicando brevemente qué principios fundamentales se abordan (procesos, planificación, memoria, sistema de archivos) y describiendo MINIX como plataforma de estudio. Debe enunciar claramente los objetivos generales del proyecto —aprender mediante la implementación práctica, modificación y análisis de componentes reales del sistema operativo— y mencionar de manera global los puntos específicos que se desarrollarán (personalización del mensaje de bienvenida, depuración de un bug en pthread, implementación del comando tree y modificación del planificador para penalizar procesos CPU-bound).

No se deben incluir resultados, detalles de implementación ni conclusiones. La extensión recomendada es de dos a tres párrafos, escrita en tono formal y técnico, que permita al profesor comprender rápidamente el alcance y la finalidad del trabajo sin necesidad de leer el resto del informe.

2. Desarrollo y resultados por componente

En esta sección se documenta, para cada uno de los puntos requeridos en el proyecto, el proceso completo de trabajo: desde el análisis inicial y las decisiones de diseño, pasando por los cambios concretos realizados en el código o la configuración del sistema, hasta la verificación experimental de su correcto funcionamiento. Se debe presentar la información de forma ordenada y por separado para cada tarea, incluyendo fragmentos de código relevantes, explicaciones técnicas de las modificaciones, dificultades encontradas y las evidencias (capturas, logs, salidas de terminal) que demuestren que el resultado obtenido cumple con los criterios de aceptación establecidos en la orientación del proyecto. El objetivo es que cualquier lector con conocimientos básicos de sistemas operativos pueda comprender qué se hizo, por qué se hizo así y cómo se comprobó su validez.

2.1. Instalación y configuración de VirtualBox y MINIX

En esta sección se debe documentar el proceso de preparación del entorno de trabajo, especificando las versiones exactas de VirtualBox y MINIX utilizadas, así como los parámetros de configuración de la máquina virtual

(memoria RAM, número de núcleos de CPU, tamaño del disco duro virtual y configuración de red). Se deben incluir las decisiones tomadas respecto a la asignación de recursos, justificándolas en función de los requisitos mínimos y recomendados del sistema. Además, se debe describir cualquier incidencia ocurrida durante la instalación (por ejemplo, problemas con el arranque, detección de discos, configuración de red o instalación de paquetes adicionales) y explicar cómo se resolvió cada una. Si se realizaron pasos adicionales no contemplados en la guía oficial (como la instalación de compiladores o herramientas de desarrollo), también deben mencionarse.

2.2. Personalización del mensaje de bienvenida

Esta sección debe explicar el proceso de modificación del archivo para que, al iniciar sesión en MINIX, se muestre un mensaje personalizado que incluya una referencia explícita a la Facultad de Matemática y Computación de la Universidad de La Habana. Se debe indicar la ruta absoluta del archivo modificado, los permisos que posee, el comando o editor utilizado para editarlo, y el contenido exacto del mensaje agregado. También se debe incluir una captura de pantalla o un volcado de terminal que evidencie que, tras el reinicio o cierre de sesión, el mensaje aparece correctamente. Si se encontraron restricciones de permisos o formatos especiales, deben documentarse.

2.3. Depuración de un bug en pthread

En esta sección debe documentar el proceso de depuración del bug en `pthread_mutex_trylock` siguiendo cuatro pasos:

- Síntomas: Explique qué ocurre al ejecutar el programa de prueba (bloqueo indefinido en la segunda llamada a `pthread_mutex_trylock`).
- Análisis: Describa la relación entre `pthread_*` (capa de compatibilidad) y `mtx_*` (implementación nativa). Identifique el archivo `libmtx/pthread_compat.c` y la función exacta donde falla. Luego, determine la causa raíz (ej. manejo incorrecto del estado del mutex, ausencia de actualización del propietario, o espera activa).
- Corrección: Presente la solución mínima propuesta, mostrando el fragmento de código modificado (preferiblemente con formato `diff`).

- Verificación: Ejecute nuevamente el programa de prueba y reporte los valores retornados por cada llamada, demostrando que ahora funcionan correctamente.

2.4. Implementación del comando `tree`

En esta sección debe documentar la implementación del comando `tree` siguiendo esta estructura:

- Diseño del algoritmo: Explique si usó recursión explícita o un enfoque iterativo con pila, y justifique por qué.
- Llamadas al sistema: Indique cuáles usó (`opendir`, `readdir`, `closedir`, `stat` o `lstat`) y para qué sirve cada una.
- Indentación: Describa cómo muestra visualmente la profundidad de cada archivo o directorio.
- Prevención de ciclos: Explique explícitamente cómo evita seguir enlaces simbólicos (ej. usando `lstat` y la macro `S_ISLNK`). Este punto es obligatorio.
- Código: Incluya los fragmentos más importantes del programa, con comentarios que expliquen su funcionamiento.
- Ejemplos de ejecución: Muestre la salida del comando en al menos tres situaciones: directorio actual (`.`), ruta absoluta (ej. `/usr`), y ruta relativa (ej. `../`). Si su programa maneja errores (permisos denegados, directorios inexistentes), también debe mostrarlo.
- Comparación: Compare la salida de su `tree` con la del comando `tree` estándar de otros sistemas (Linux, por ejemplo) y demuestre que cumple los requisitos.

2.5. Penalización por uso intensivo de CPU

Esta es la sección más extensa y debe estructurarse en subapartados claros.

- **Análisis previo:** Identifique los archivos del scheduler donde se gestiona el quantum (ej. `do_noquantum`), las estructuras de datos de planificación, y cómo se representan prioridad y quantum.
- **Diseño de la política:** Defina los parámetros elegidos (tamaño de ventana, valor de N, criterios de penalización y recuperación) y justifíquelos.
- **Cambios realizados:** Muestre los fragmentos de código modificados, explicando línea por línea su funcionamiento.
- **Validación experimental:** Describa dos escenarios de prueba (CPU-bound vs. interactivo) y presente la evolución de sus prioridades con capturas o tablas.

3. Resultados globales y discusión

En esta sección se resume el estado final de cada componente del proyecto, indicando explícitamente si funciona completamente, parcialmente o no funciona, y se discuten las dificultades más significativas encontradas durante la implementación. Se debe analizar la coherencia entre los resultados obtenidos y los esperados según la orientación del proyecto, explicando posibles desviaciones y sus causas. Además, se reflexiona sobre las implicaciones de las decisiones de diseño adoptadas y se extraen lecciones aprendidas sobre el funcionamiento interno de los sistemas operativos, especialmente en aspectos como la planificación de procesos, la concurrencia y el acceso al sistema de archivos mediante llamadas al sistema.

4. Conclusiones

En esta sección se evalúa el cumplimiento de los objetivos planteados en la introducción, destacando qué principios de sistemas operativos (procesos, planificación, memoria, sistema de archivos) fueron efectivamente comprendidos y aplicados. Se debe valorar el grado de éxito alcanzado en cada una de las actividades concretas realizadas. Finalmente, se proponen posibles mejoras o extensiones al trabajo realizado y se reflexiona sobre la utilidad de la experiencia para la formación en Ciencia de la Computación.

5. Referencias consultadas

Liste las fuentes utilizadas durante el proyecto. Pueden ser: libros de texto (Tanenbaum), documentación oficial de MINIX, tutoriales en línea, repositorios de código u otras. Use un formato consistente (ej. *ISO 690* o *APA*).

6. Declaración de uso de IA

Declaración explícita y honesta sobre el uso de herramientas de inteligencia artificial generativa (como ChatGPT, Copilot u otra) durante el desarrollo del proyecto. Se debe especificar:

- Partes del proyecto donde se usó IA: (ej. depuración del bug en `pthread`, redacción de esta plantilla)
- Naturaleza del uso: (ej. generación de fragmentos de código, corrección de errores, mejora de redacción técnica, explicación de conceptos)

7. Anexos (si aplica)

- Enlace a repositorio.
- Logs de ejecución de las pruebas.
- Capturas de pantalla del sistema funcionando.